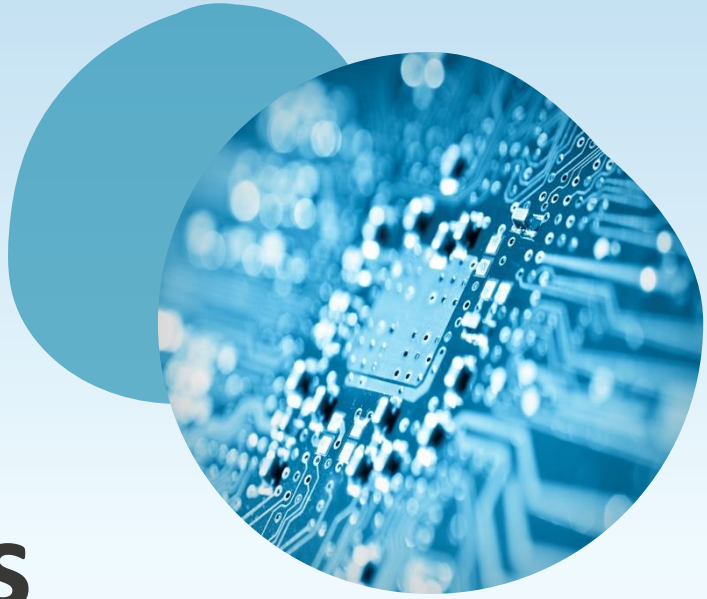


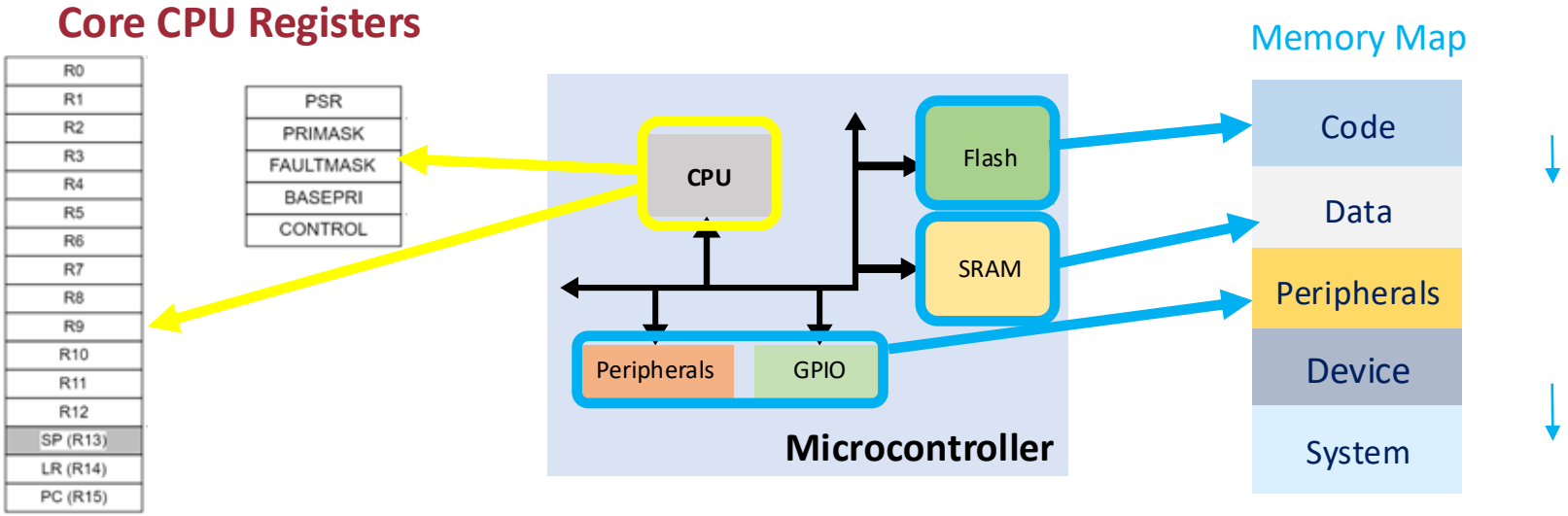
MSP432 Memory Mapped I/O Details

Embedded Software for the Internet Of Things

Kasim Sinan Yildirim

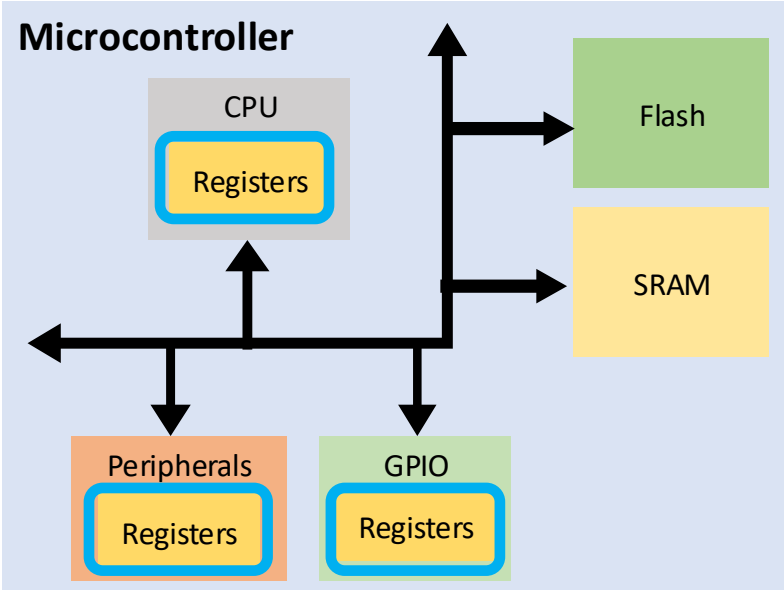
Memory Map

- **Memory Map:** Provides a memory address to physical device mapping within an address space for use in programming



Register Memory

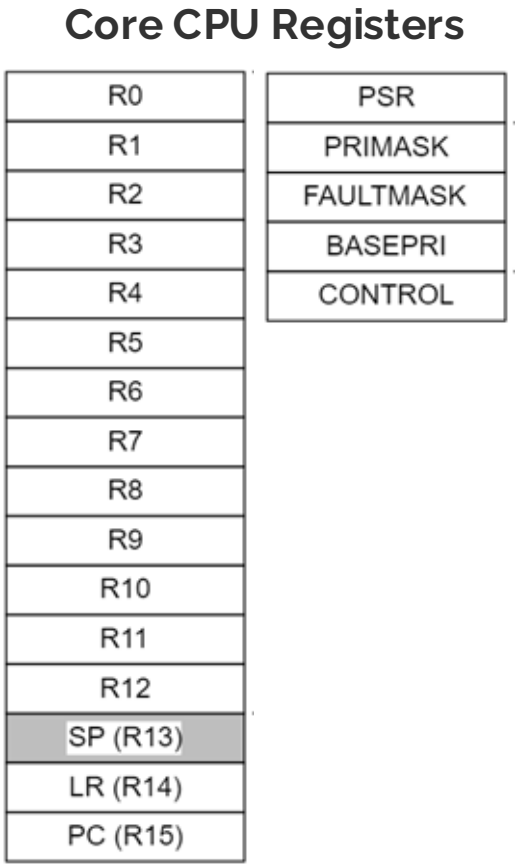
- Registers from Peripherals, System and Vendor-specific regions have **immutable** addresses
  - Typical Register Memories
  - Internal Core CPU
  - Internal Private Peripherals
  - External Private Peripherals
  - General Peripheral Memory



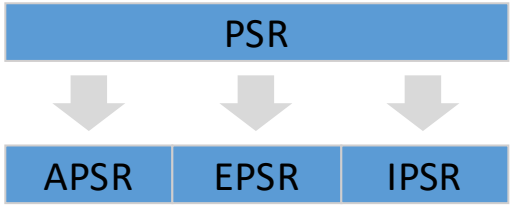
Core CPU Registers

- Cortex-M CPU Core registers
  - r0-r15 General Purpose
  - r13-r15 Special Role (lr, sp, pc)
  - Program Status Registers
  - Exception Mask Registers
  - Control Register
- General Purpose Registers
  - Used on every clock cycle
  - Always updating

Not part of the memory map. Needs special instructions to read/write



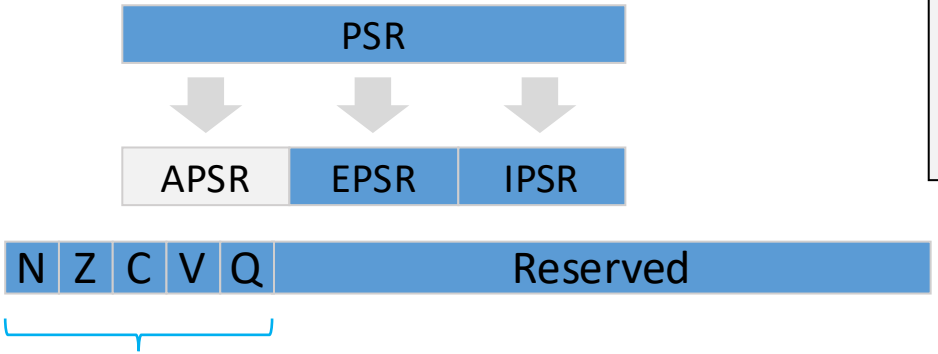
CORTEX-M4 Program Status Registers



The **Program Status Register (PSR)** captures the current CPU and program application state.

The PSR combines the APSR, EPSR, IPSR

CORTEX-M4 Program Status Registers



The **Application Program Status Register (APSR)** contains current state of condition flags from *previous instruction execution*

N = Negative Flag → Was last result negative?

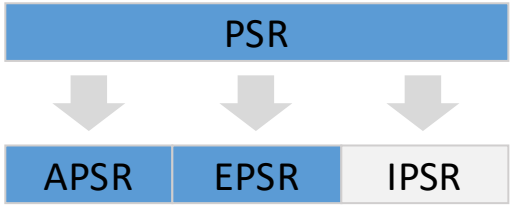
Z = Zero Flag → Was last result zero?

C = Carry Flag → Did last result have a carry?

V = Overflow Flag → Did last result have an overflow?

Q = Saturation Flag → Did last result saturate?

CORTEX-M4 Program Status Registers



The **Interrupt Program Status Register (IPSR)** contains the *exception type number* of the current ISR.



Number of Current Exception

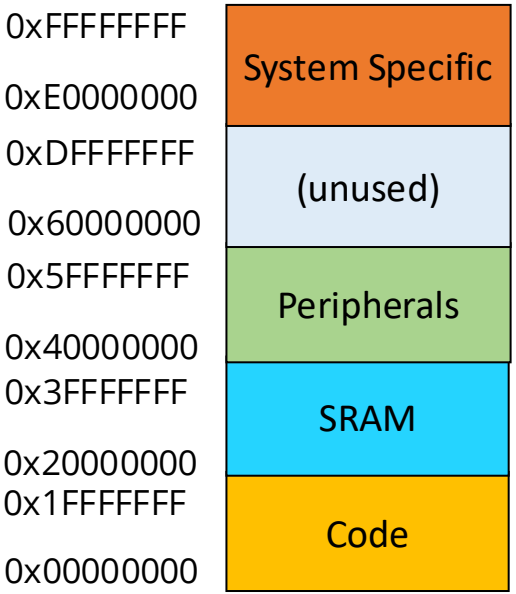
Exception Numbers:

0 = Reset
1 = Non-Maskable Interrupt (NMI)
2 = Hard Fault
3 = Memory Management Fault
…
16 = IRQ0
17 = IRQ1
…
N+15 = IRQ(n-1)

Lab#1: IPSR Register

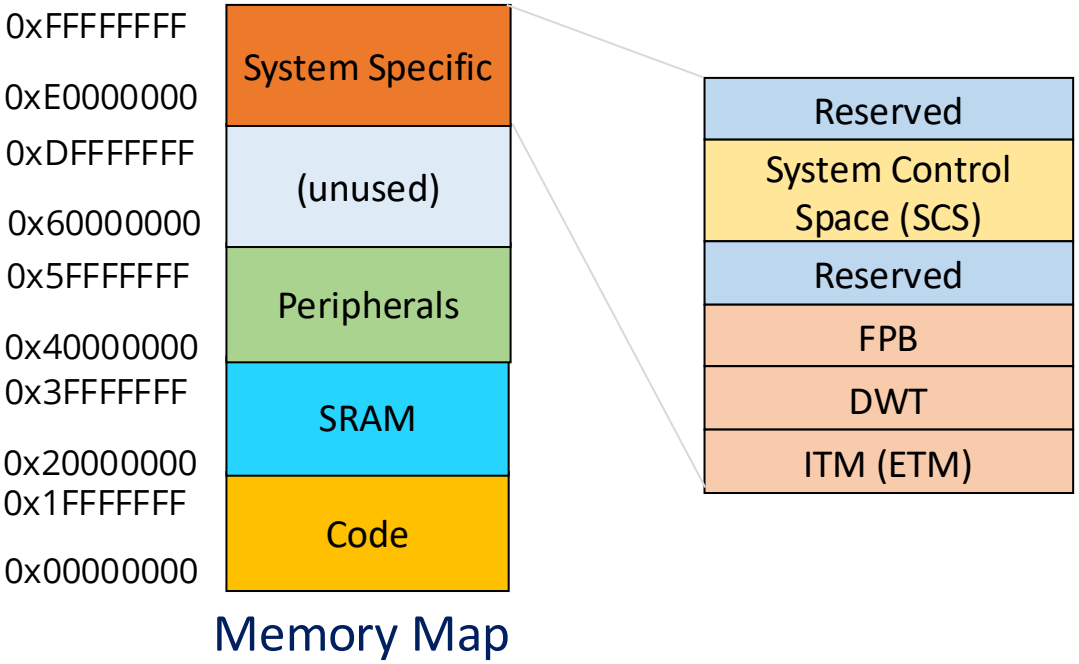
- Check the fields of the IPSR when button press and timer interrupts occur.

Private Peripherals



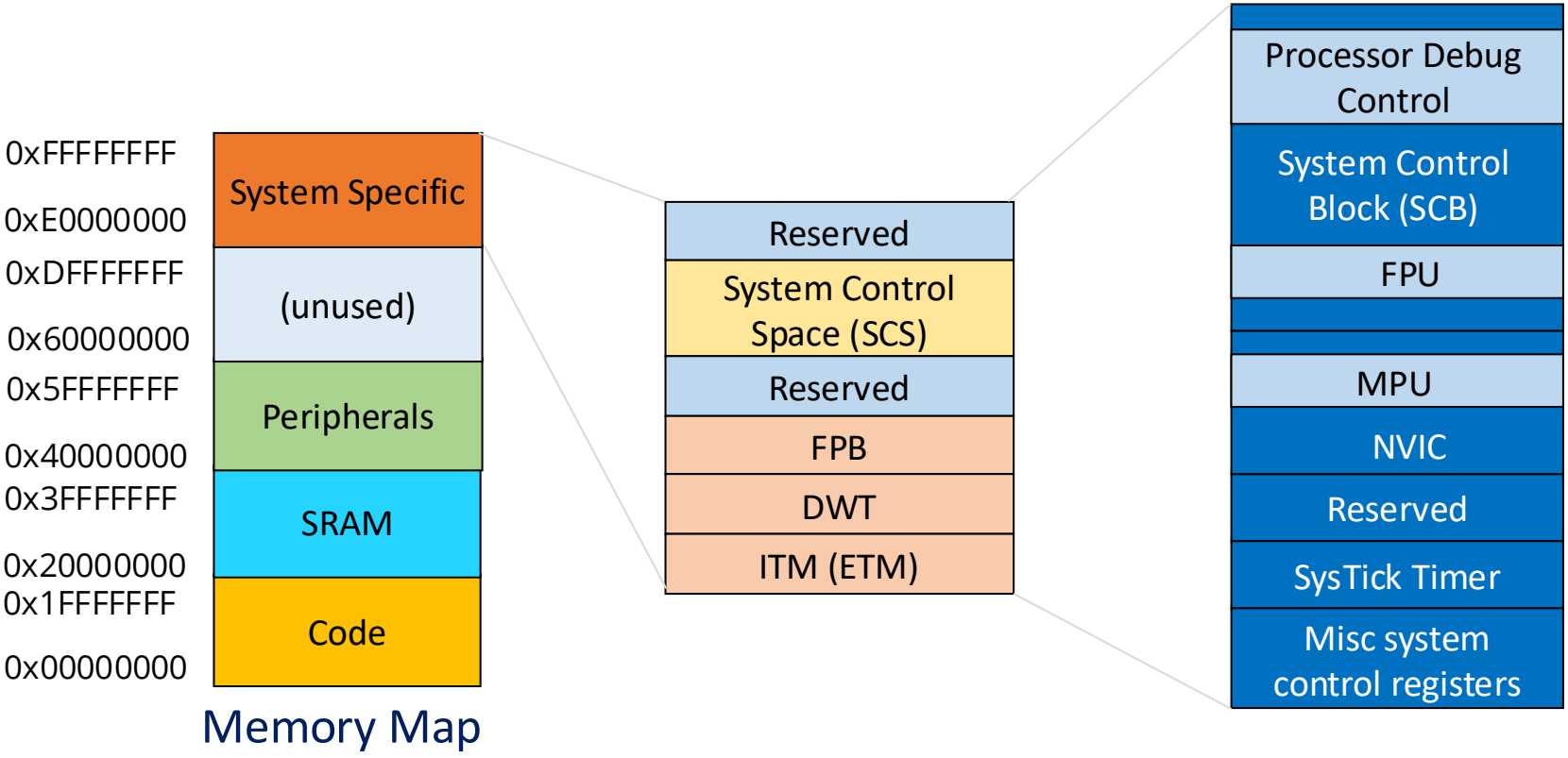
Memory Map

Private Peripherals

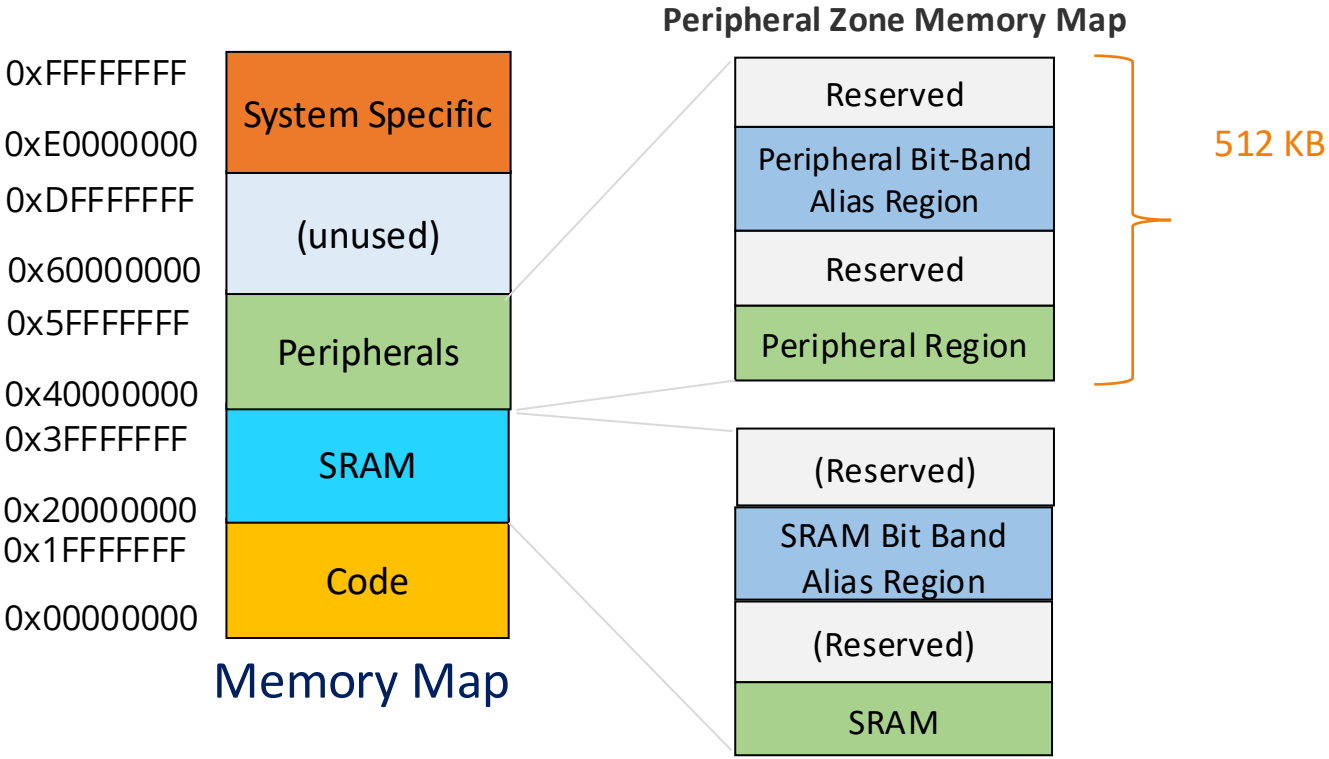


Memory Map

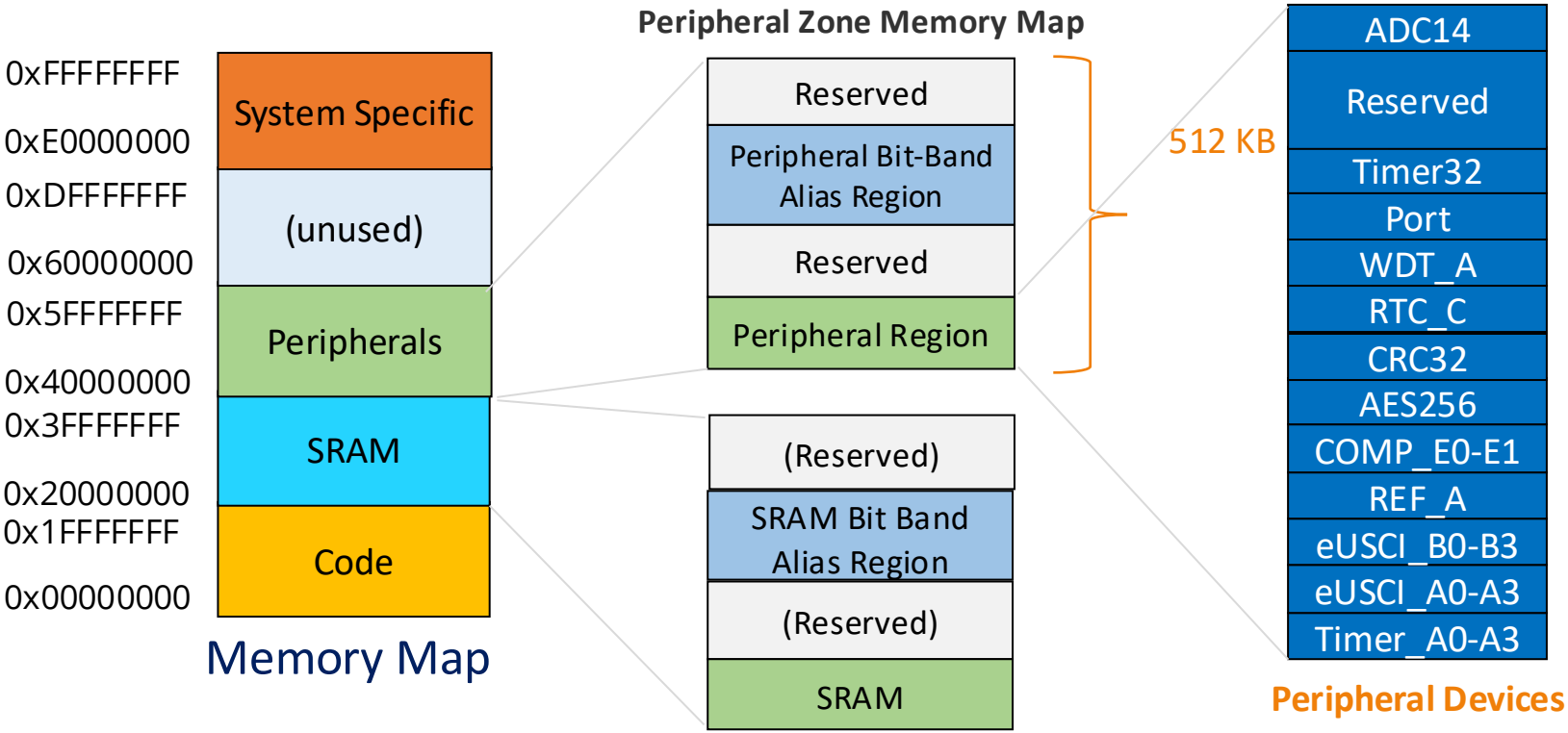
Private Peripherals



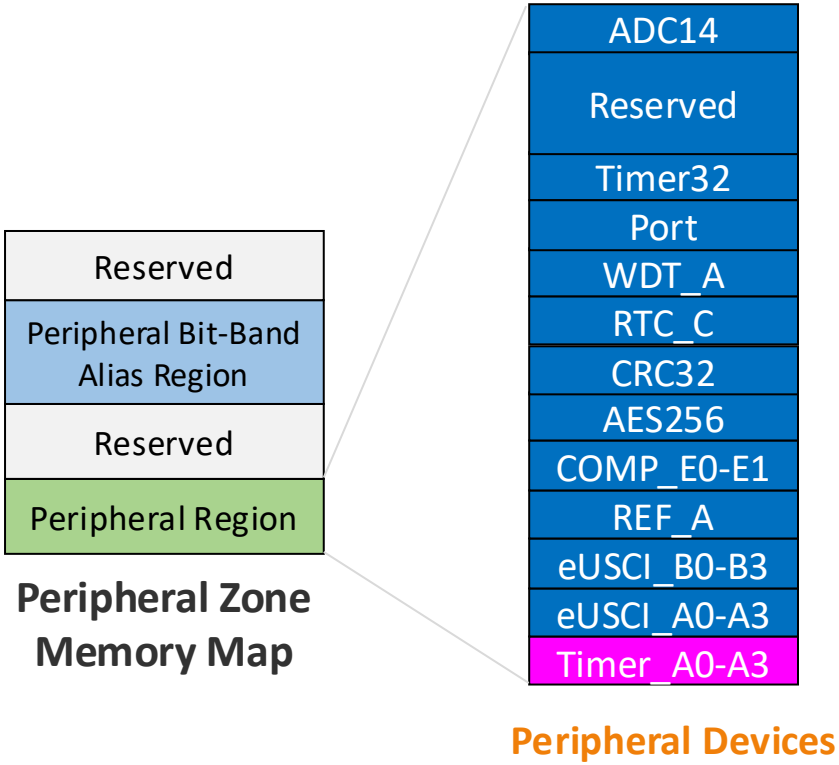
General Peripherals



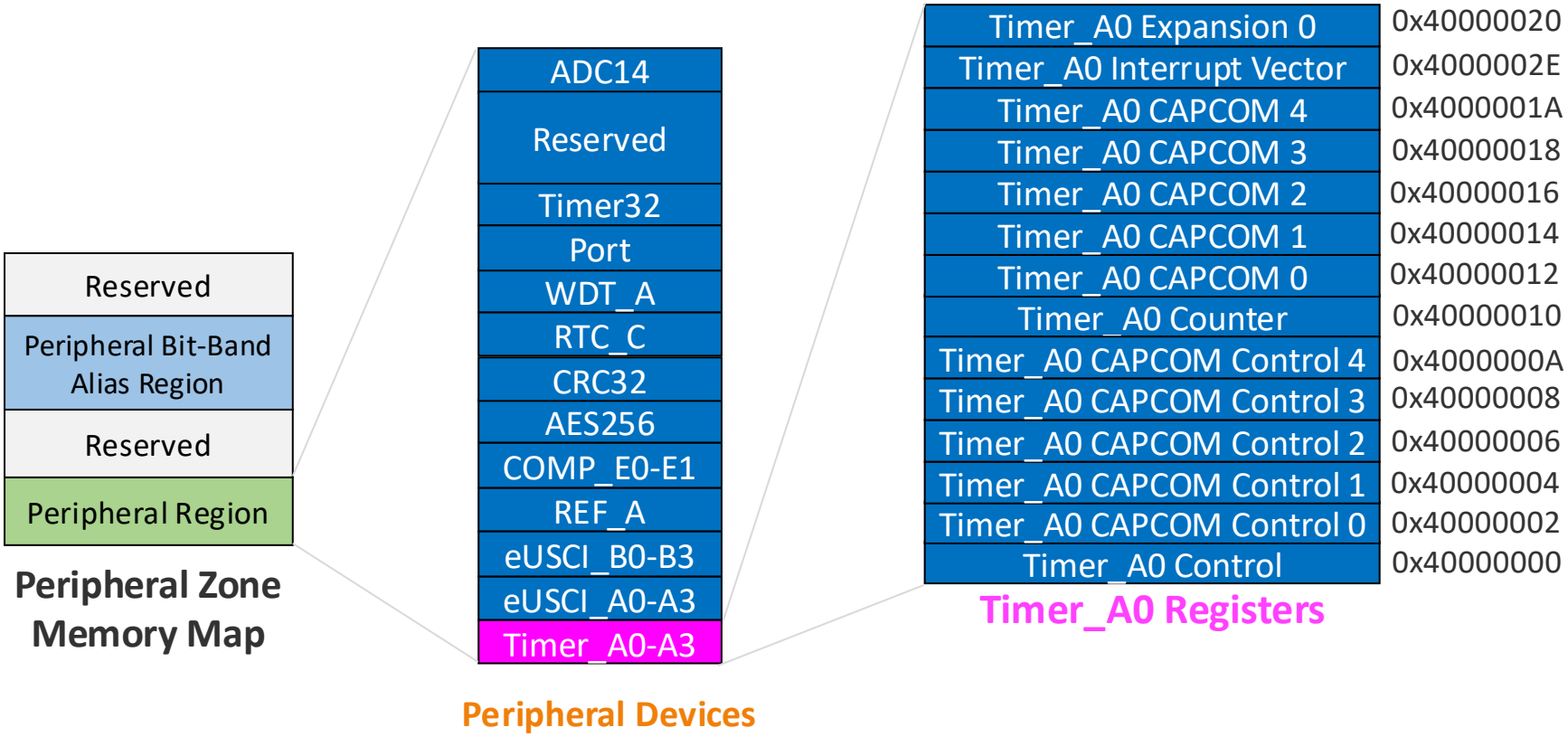
General Peripherals



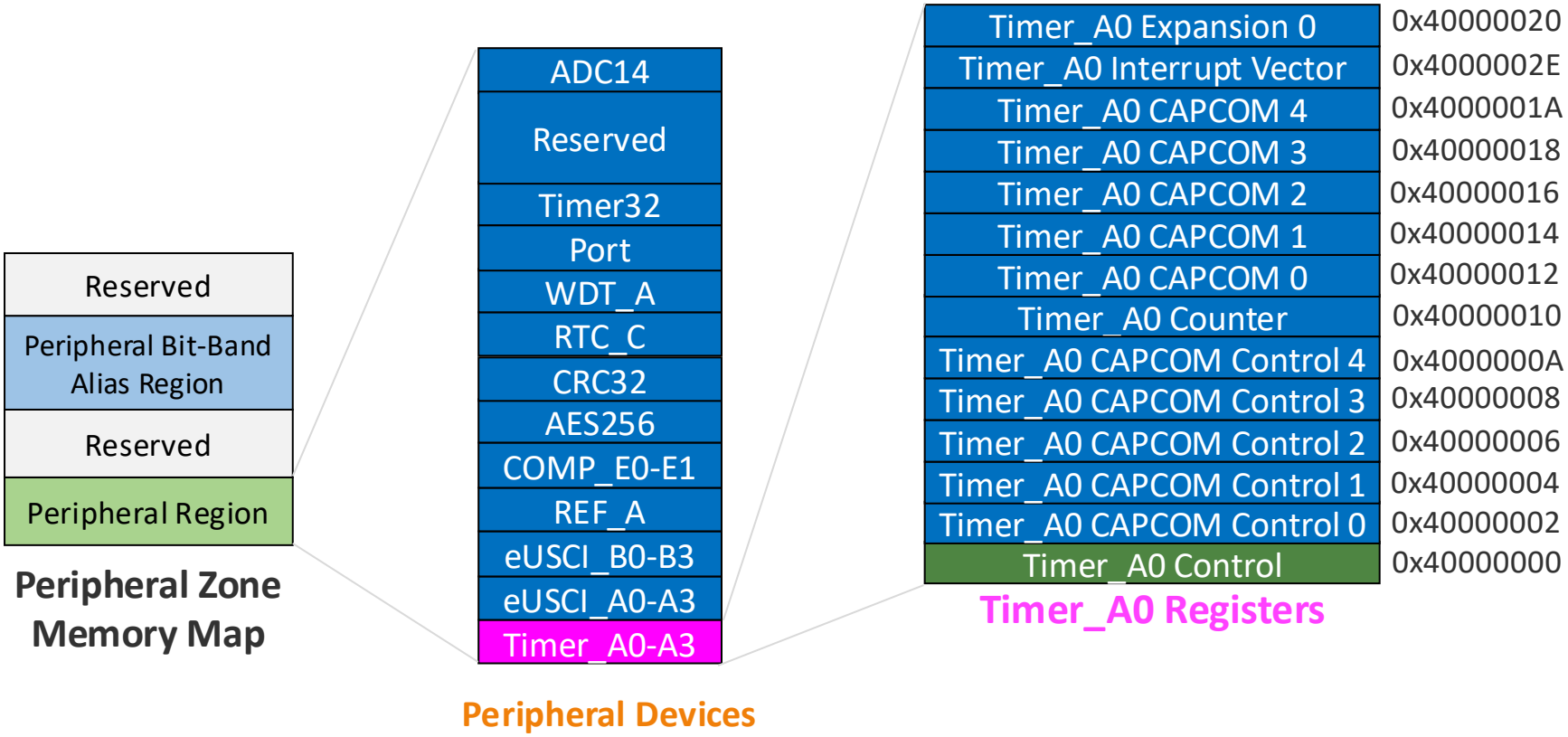
Timer A0 Peripheral



Timer A0 Peripheral

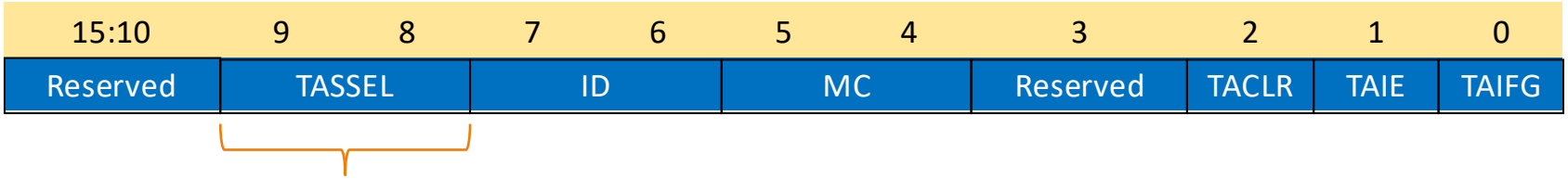


Timer A0 Peripheral



TIMER A0 Control Register

Timer_A0 Control Register



Bit fields are variables with *predefined widths* (i.e. non-standard sizes) within the register

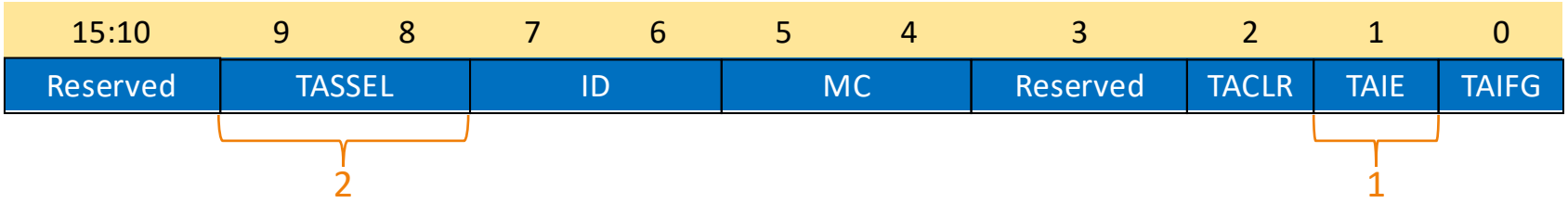
Bit Fields help represent operations/status/configurations.

Bit fields of Timer_A0 Control Register

TASSEL	= Select input clock source for Timer_A0
ID	= Select input clock divider for Timer_A0
MC	= Select count-mode for Timer_A0
TACLR	= Resets Timer_A0
TAIE	= Enable Timer_A0 interrupt
TAIFG	= Timer_A0 interrupt flag

TIMER A0 Control Register

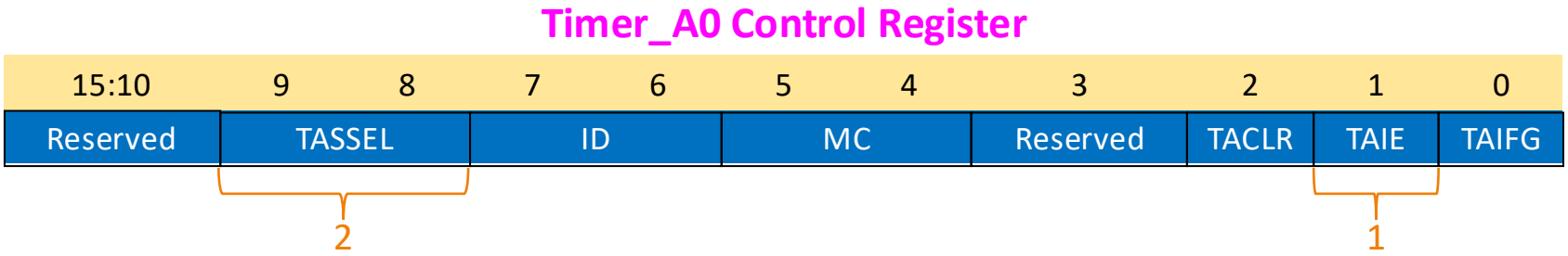
Timer_A0 Control Register



16-Bit Register (Address = 0x40000000), configure for: TASSEL = 2, and TAIE = 1

```
volatile uint16_t * ta0_ctrl = (uint16_t*)0x40000000;
*ta0_ctrl = 0x0202;
```

TIMER A0 Control Register



16-Bit Register (Address = 0x40000000), configure for: TASSEL = 2, and TAIE = 1

```
(*(volatile uint16_t*)0x40000000)) = 0x0202;
```

Dereference!

Equivalent to Defining a Pointer!

Not Very Readable or Maintainable…

Memory Access MACROs

- Use preprocessor to define an access method without using hardcoded values

```
/ * 8, 16, & 32 Bit Register Access Macros */
#define HWREG8(x)       (*((volatile uint8_t  *)(x)))
#define HWREG16(x)      (*((volatile uint16_t *)(x)))
#define HWREG32(x)      (*((volatile uint32_t *)(x)))

#define TA0CTL                  (HWREG16(0x40000000))
```

Pass in **any address** to access a Register

Example Use of Access Macro:

```
TA0CTL = 0x0202;
```



```
(*(volatile uint16_t*)0x40000000)) = 0x0202;
```

Timer A0 Registers

```
#define HWREG16(x)     (*((volatile uint16_t *)(x)))

/* Example Use of Access Macros */
#define TA0CTL          (HWREG16(0x40000000))
#define TA0CCTL0        (HWREG16(0x40000002))
#define TA0CCTL1        (HWREG16(0x40000004))
#define TA0CCTL2        (HWREG16(0x40000006))
#define TA0CCTL3        (HWREG16(0x40000008))
#define TA0CCTL4        (HWREG16(0x4000000A))
#define TA0R            (HWREG16(0x40000010))
#define TA0CCR0         (HWREG16(0x40000012))
#define TA0CCCR1        (HWREG16(0x40000014))
#define TA0CCCR2        (HWREG16(0x40000016))
#define TA0CCCR3        (HWREG16(0x40000018))
#define TA0CCCR4        (HWREG16(0x4000001A))
#define TA0EX0          (HWREG16(0x40000020))
#define TA0IV           (HWREG16(0x4000002E))
```

Timer_A0 Interrupt Vector	0x4000002E
Timer_A0 Expansion 0	0x40000020
Timer_A0 CAPCOM 4	0x4000001A
Timer_A0 CAPCOM 3	0x40000018
Timer_A0 CAPCOM 2	0x40000016
Timer_A0 CAPCOM 1	0x40000014
Timer_A0 CAPCOM 0	0x40000012
Timer_A0 Counter	0x40000010
Timer_A0 CAPCOM Control 4	0x4000000A
Timer_A0 CAPCOM Control 3	0x40000008
Timer_A0 CAPCOM Control 2	0x40000006
Timer_A0 CAPCOM Control 1	0x40000004
Timer_A0 CAPCOM Control 0	0x40000002
Timer_A0 Control	0x40000000

Structure Overlay

- Define a C structure to directly match peripheral region registers

```
typedef struct {
 __IO   uint16_t CTL;
 __IO   uint16_t CCTL[5];
        uint16_t RESERVED0[2];
 __IO   uint16_t R;
 __IO   uint16_t CCR[5];
        uint16_t RESERVED0[2];
 __IO   uint16_t EX0;
        uint16_t RESERVED0[6];
 __I    uint16_t IV;
} Timer_A_Type;
```

```
/* Define the Base Address of Peripheral Regions */
#define PERIPH_BASE       ((uint32_t) 0x40000000)
#define TIMER_A0_BASE     (PERIPH_BASE + 0x00000000)
#define TIMER_A1_BASE     (PERIPH_BASE + 0x00000400)
#define TIMER_A2_BASE     (PERIPH_BASE + 0x00000800)

/* Multiple Timer Modules, different Addresses */
#define TIMER_A0          ((Timer_A_Type *) TIMER_A0_BASE)
#define TIMER_A1          ((Timer_A_Type *) TIMER_A1_BASE)
#define TIMER_A2          ((Timer_A_Type *) TIMER_A2_BASE)
```

```
#define __IO (volatile)
#define __I  (volatile const)
```

Example Use of Structure Overlay:

```
TIMER_A0->CTL = 0x0202;
```

Structure Overlay

- Structure Overlays require exact replica of peripheral region
  - Size registers to equivalent standard types
  - Order Matters
  - Leave space for reserved bytes
  - Read-Only Registers = const!
  - All registers are volatile

```
typedef struct {
  __IO   uint16_t CTL;
  __IO   uint16_t CCTL[5];
         uint16_t RESERVED0[2];
  __IO   uint16_t R;
  __IO   uint16_t CCR[5];
         uint16_t RESERVED0[2];
  __IO   uint16_t EX0;
         uint16_t RESERVED0[6];
  __I    uint16_t IV;
} Timer_A_Type;
```

Unused memory in the peripheral region

```
#define __IO (volatile)
#define __I  (volatile const)
```

Bitwise Operators

- C-programming provides bitwise operators
  - << >> & | ^ ~

```
uint8_t * ptr = (uint8_t *)0x1000;

*ptr |= 0x10;
*ptr &= ~(0x10);
*ptr ^= 0x10;
```

Set 4th bit:

Clear 4th bit:

Toggle 4th bit:

Bitwise Operators

- C-programming provides bitwise operators
  - << >> & | ^ ~

```
uint8_t * ptr = (uint8_t *)0x1000;

*ptr |= 0x10;
*ptr &= ~(0x10);
*ptr ^= 0x10;
```

Set 4th bit:
Clear 4th bit:
Toggle 4th bit:

All bits preserved **except** bit 4

Combine logic with assignment, performs a read, modify, write

Bit Masks

- Bit Masks are constant expressions used to set, clear, or toggle a specific set of bits

```
foo |= 0x30 ;
foo &= ~(0xC0);
foo ^= 0x0E ;
```

Bit Masks

- Bit Masks are constant expressions used to set, clear, or toggle a specific set of bits

```
foo |= (BIT4 | BIT5);
foo &= ~( BIT7 | BIT6 );
foo ^= (BIT3 | BIT2 | BIT1);
```

```
#define BIT0    (uint16_t)(0x0001)
#define BIT1    (uint16_t)(0x0002)
#define BIT2    (uint16_t)(0x0004)
#define BIT3    (uint16_t)(0x0008)
#define BIT4    (uint16_t)(0x0010)
#define BIT5    (uint16_t)(0x0020)
#define BIT6    (uint16_t)(0x0040)
#define BIT7    (uint16_t)(0x0080)
#define BIT8    (uint16_t)(0x0100)
#define BIT9    (uint16_t)(0x0200)
#define BITA    (uint16_t)(0x0400)
#define BITB    (uint16_t)(0x0800)
#define BITC    (uint16_t)(0x1000)
#define BITD    (uint16_t)(0x2000)
#define BITE    (uint16_t)(0x4000)
#define BITF    (uint16_t)(0x8000)
#define BIT(x)  ((uint16_t)1 << (x))
```